# Pipeline Parallelism for CPU Inference on a Virtual Rocks Cluster

Mihai Munteanu

munteanum222@yahoo.com

https://github.com/MihaiMunteanu1/Pipeline-Parallel-Inference-MPI

**Abstract**

Large neural networks are often split across machines, but on a small CPU cluster it is not obvious when the split pays for itself: every stage is slow, the network adds latency, and a single request still has to visit all the stages. This paper measures pipeline parallelism for neural inference on a virtual Rocks 6.2 cluster running on one laptop, with one head node and up to four compute nodes. Because CentOS 6 cannot run current machine-learning libraries, the models are executed by C programs that place consecutive layers on different nodes and move the intermediate tensors with Open MPI. Three models are studied in order: a small Llama-style text generator and two BERT-style sentence encoders, of 22.6 and 108.9 million parameters. The outputs match the reference implementations, with a difference of at most $2 \times 10^{-7}$ for the largest encoder. Generating one sequence becomes 23% slower when split, because each token waits for the previous one, and an even split of the layers gains nothing even with two sequences, since the model's final classifier costs more than all its layers together. Encoding independent texts behaves differently. With `bge-base-en-v1.5`, throughput rises by 1.39 times on two nodes and 2.16 times on four, an efficiency of 54%. The gain comes from keeping the nodes busy at the same time: with the smaller encoder on two nodes, one text in flight is 9% slower than a single node, while four texts in flight are 53% faster, and larger windows add little. On four nodes, each rank spends about a quarter of its time in MPI calls, and the total computing time also seems to grow when the virtual machines run together. The system supports live semantic search over 1,000 news texts, with a question embedded in about 1.4 seconds on four nodes, about twice as long as on one, and a live view shows the nodes working simultaneously. The main limits of these results are that all virtual machines share one laptop and that each configuration was run only three or four times.

## 1 Introduction

Pipeline parallelism splits a network into consecutive stages, one per device, and passes intermediate results from stage to stage [1, 2]. It is widely used to fit large models on several devices and to raise throughput. Its value on a small CPU cluster is less clear. The stages take long, the network adds latency, and a single request has to visit every stage anyway.

This work tests that trade-off on a deliberately modest platform: a virtual cluster installed with Rocks [3] in VirtualBox, on a laptop with 14 CPU cores and 16 GB of RAM. The platform also imposes a hard constraint. Rocks 6.2 is based on CentOS 6.6, with glibc 2.12 and GCC 4.4.7, so PyTorch and current releases of llama.cpp [4] cannot run on it. The models were therefore executed by small C programs, with MPI [5, 6] as the only communication layer.

The contributions are:

- a C implementation of pipeline-parallel inference that runs on a dated HPC stack, with its outputs checked against reference implementations;
- measurements of three models on up to four nodes, including a case where splitting makes inference slower;
- an operation-count analysis that explains why an even split of the layers can be the wrong split;
- a live view and a working semantic-search demo, which show the pipeline at run time.

## 2 Background and Related Work

### 2.1 Clusters, head nodes and compute nodes

A cluster is a group of networked computers that work on one problem. In a Rocks cluster [3], the *head node* (frontend) installs and administers the machines and exports shared folders. The *compute nodes* do the calculation. They are installed automatically over the network: `insert-ethers` on the head node registers each new machine, which then boots through PXE and installs itself. Optional software comes as *rolls*, such as `hpc`, which provides Open MPI, and `sge`, which provides a batch scheduler.

### 2.2 MPI

MPI [5] is a standard for processes that do not share memory and exchange explicit messages. `mpirun` starts the same program on the nodes listed in a host file, and every process learns its identity, or *rank*, from the library. This work uses Open MPI [6] and a small part of the interface: blocking `MPI_Send` and `MPI_Recv`, non-blocking `MPI_Isend` with `MPI_Test`, `MPI_Iprobe` to check for arriving messages, and `MPI_Gather` to collect measurements.

### 2.3 Pipeline parallelism

Consider a model cut into $N$ stages, where stage $i$ takes time $t_i$ and sends its output in time $c_i$. One item must cross all stages, so its latency cannot improve, while the steady-state throughput is limited by the slowest stage:

$$L = \sum_{i=1}^{N} (t_i + c_i), \qquad X \approx \frac{1}{\max_i (t_i + c_i)}. \tag{1}$$

Throughput improves only if several items are in the pipeline at once. Figure 1 shows the difference. As with Amdahl's law [7], the part that cannot be divided sets the limit.

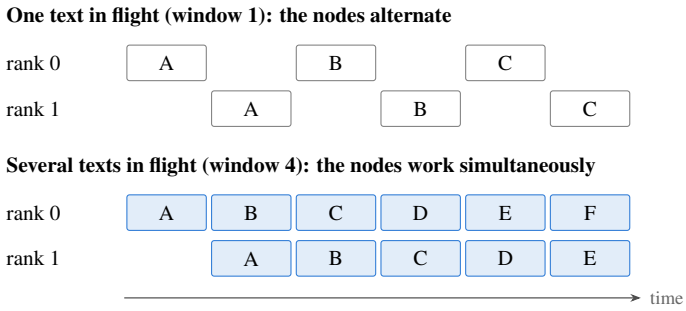



**Figure 1:** Execution of a two-stage pipeline (schematic, not to scale). Each cell is half of the model run on one text.

### 2.4 Transformer encoders

The encoders used here are BERT models [8], built from the transformer of Vaswani et al. [9]. Text is split into tokens, each represented by a vector of $d$ numbers, giving a matrix $x \in \mathbb{R}^{n \times d}$ for $n$ tokens. A layer first applies multi-head self-attention. For each of the heads, of size $d_h$, the scores between queries $Q$ and keys $K$ select how much of the values $V$ each token takes from the others:

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^{\top}}{\sqrt{d_h}}\right)V. \tag{2}$$

The layer then passes each token through a two-matrix network with the GELU activation [10]. Each sub-layer has a residual connection and layer normalization [11]:

$$x' = \mathrm{LN}\big(x + W_o\,\mathrm{Attn}(xW_q, xW_k, xW_v)\big), \tag{3}$$

$$x'' = \mathrm{LN}\big(x' + W_2\,\mathrm{GELU}(W_1 x')\big). \tag{4}$$

Since the output of a layer has the same shape as its input and is the only input of the next layer, the model can be cut between any two layers (Figure 2). Sentence-embedding systems [12] reduce the matrix to one vector, by averaging the tokens or by taking the first one, and normalize it. Two texts are then compared by cosine similarity, $\cos(u, v) = u^\top v$ for unit vectors.

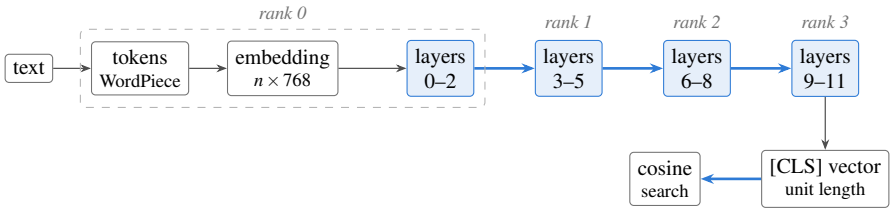



**Figure 2:** The encoder cut across four ranks. Blue arrows are MPI messages: an $n \times 768$ matrix between layers and one 768-dimensional vector back to rank 0.

### 2.5 Related work

GPipe [1] partitions a model across accelerators for training, and PipeDream [2] improves the balance and utilization of such pipelines, also for training. Petals [13] serves large models from blocks spread over the machines of many participants. On a single server, Orca [14] schedules requests per iteration, because text generation repeats its forward pass for every token. This work studies a much smaller setting, CPU inference on a virtual cluster, and focuses on how the task type and the split decide whether pipelining helps. The generation experiments build on llama2.c [15], a single-file C implementation of Llama 2-style inference [16].

## 3 System

### 3.1 The virtual cluster

The cluster runs on one laptop (Intel i9-13900H, 14 cores, 16 GB of RAM) under VirtualBox (Figure 3). Each compute node has 2 GB of RAM and one virtual CPU, and the head node has 2 GB and two, so ten of the 16 GB are allocated. An internal network connects the nodes, and only the head node has a second, bridged adapter for internet access. The home folder is shared, so a program compiled on the head node, its model and its data are visible on every node. Rocks installed the first two compute nodes, and the other two were added later with `insert-ethers`. Table 1 lists the software.

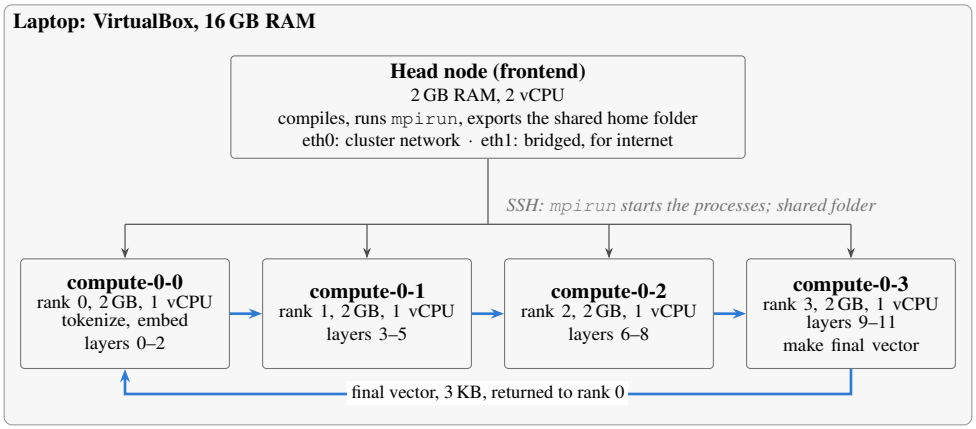



**Figure 3:** The cluster with four compute nodes. Head node and compute nodes share a folder. Model data (blue) moves directly between compute nodes through MPI over the internal network.

| Component | Version | Component | Version |
|---|---|---|---|
| Rocks | 6.2 | GCC | 4.4.7 |
| Operating system | CentOS 6.6 | Open MPI | 1.6.2 |
| glibc | 2.12 | Installed rolls | kernel, hpc, os, base, sge |

**Table 1:** The software stack on the cluster.

### 3.2 Models

Table 2 lists the three models, in the order they were studied. `stories15M` is a small Llama 2-style generator from llama2.c, trained on TinyStories [17]. `all-MiniLM-L6-v2` is a compact sentence encoder based on MiniLM [18]. `bge-base-en-v1.5` [19] is a larger encoder, chosen because its 12 layers divide evenly among 2, 3 and 4 nodes. For encoders, the program was written from the architecture; weights and vocabulary were exported from Hugging Face in Google Colab into a flat binary file.

| | `stories15M` | `all-MiniLM-L6-v2` | `bge-base-en-v1.5` |
|---|---|---|---|
| Task | generation | embedding | embedding |
| Layers | 6 | 6 | 12 |
| Dimension $d$ | 288 | 384 | 768 |
| Attention heads | 6 | 12 | 12 |
| Feed-forward width $h$ | 768 | 1,536 | 3,072 |
| Vocabulary | 32,000 | 30,522 | 30,522 |
| Parameters (millions) | 15 | 22.6 | 108.9 |
| Weights, float32 (MB) | 58.0 | 86.1 | 415.4 |
| Multiplications per layer and token | 1.00 M | 1.77 M | 7.08 M |
| Final step per token | classifier, 9.22 M | none | none |
| Output vector | – | mean of tokens | [CLS] token |

**Table 2:** The three models. Multiplications count matrix products only: $4d^2 + 3dh$ per Llama layer, $4d^2 + 2dh$ per BERT layer, and $d$ times the vocabulary for the classifier.

### 3.3 The pipeline

Each rank maps the model file and uses only its own layers, so unused pages are never loaded. For encoders, rank 0 tokenizes the text, computes the embeddings and runs its layers. The matrix then goes from rank to rank, and the last rank builds the vector and returns it to rank 0. For the generator, the model's classifier shares its weights with the embedding table, so rank 0 also computes the logits and samples the next token. This gives the pipeline an extra expensive stage.

A window limits the number of texts in flight (`-w`). Rank 0 starts a new text while the window has room and otherwise waits for a result:

```
if (next < n && next - done < window) {
   /* tokenize, embed and run the local layers, then send */
} else {
   MPI_Recv(out + (size_t)done * d, d, MPI_FLOAT, size - 1, TAG_DATA,
          MPI_COMM_WORLD, MPI_STATUS_IGNORE);
   done++;
}
```

Every other rank repeats the same loop. Messages carry no size header: the receiver reads the number of tokens from the message itself, since texts have different lengths.

```
MPI_Recv(x, m->max_len * d, MPI_FLOAT, rank - 1, MPI_ANY_TAG, MPI_COMM_WORLD, &st);
if (st.MPI_TAG == TAG_STOP) break;
MPI_Get_count(&st, MPI_FLOAT, &count);
int n = count / d;
forward_layers(m, x, n);
```

These ranks form a ring, so blocking sends could wait on each other indefinitely. To exclude this, rank 0

never blocks while sending. It uses `MPI_Isend`, and until the send completes it keeps receiving finished vectors:

```
MPI_Isend(x, len * d, MPI_FLOAT, 1, TAG_DATA, MPI_COMM_WORLD, &req);
int sent = 0;
while (!sent) {
   MPI_Test(&req, &sent, MPI_STATUS_IGNORE);
   if (!sent) try_receive(out, d, size, &done);
}
```

With `-t 1`, every rank prints what it processed, with which layers, when, and what it sent (Figure 8).

# 4 Evaluation Method

## 4.1 Data

The encoders process the first 1,000 texts of the AG News test set [20], which has news in English from four categories, for a total of 54,458 tokens. To keep the runs short, measurements use the first 200 texts (11,671 tokens) for MiniLM and the first 50 (3,137 tokens) for BGE. The generator produces at most 200 tokens per sequence, with a fixed seed.

## 4.2 Metrics

Throughput is in texts or tokens per second. With $T_1$ the mean time on one node and $T_N$ the mean time on $N$ nodes,

$$S = \frac{T_1}{T_N}, \qquad E = \frac{S}{N}. \tag{5}$$

Each rank also reports its computing time, and the time inside MPI calls, which includes both transfers and idle waiting. Peak memory is the `VmHWM` value of the process. All measurements were taken on the cluster, with each configuration run three or four times.

## 4.3 Correctness

Speed is only meaningful if the split does not change the output. Vectors from the C encoder were compared with `transformers` in Python, on six check texts. The largest difference per component was $1 \times 10^{-7}$ for MiniLM on two nodes and $2 \times 10^{-7}$ for BGE on four nodes. With random weights and a test vocabulary, token sequences matched on 139 texts, and vectors were identical across 1, 2, 3 and 4 processes. For the generator, the text produced on two nodes was byte-identical to the single-process output with the same seed.

# 5 Results

## 5.1 Generation: splitting can make things slower

Figure 4 shows `stories15M` on two nodes. With one sequence, splitting costs 23%: each token depends on the previous one, so only one node works at a time and the network is pure overhead. Two simultaneous sequences allow concurrency, but the even split still gains nothing. Only moving all layers off rank 0 gives a gain, of 23%.

Operation counts explain this. Table 3 gives each rank's work per token. The classifier, with 9.22 million multiplications, costs more than all six layers (5.97 million). An even split therefore leaves rank 0 with most of the work, and the slowest stage limits throughput. With a 0+6 split the bound is 1.65 against 1.24. The measured rank times follow the same pattern: for 0+6, rank 0 computed for 10.03 s and rank 1 for 5.99 s of the 11.99 s total. The practical point is that *a split should balance work, not layer counts*.

## 5.2 Embedding: independent texts keep nodes busy

The texts given to an encoder do not depend on each other, and BERT has no expensive last stage, so a 3+3 split of MiniLM is balanced. Figure 5 varies the window on two nodes. With one text in flight, two nodes are

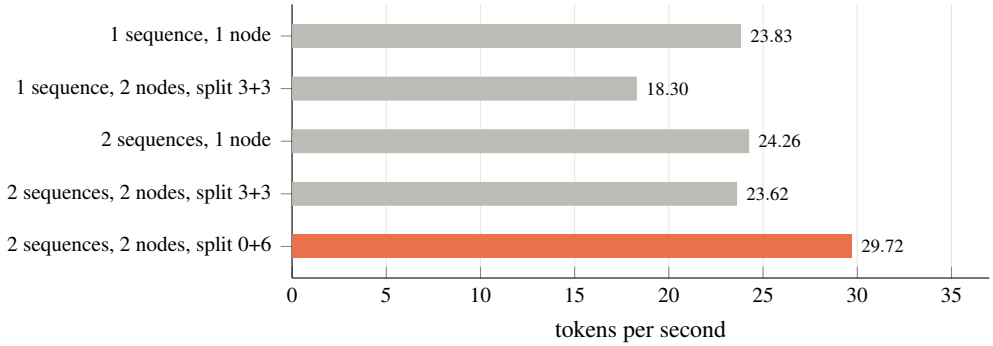



**Figure 4:** Generation with `stories15M`, mean of three runs. Even splitting does not help; moving the layers away from the classifier does.

| Split of `stories15M` | Rank 0 (M) | Rank 1 (M) | Bound $\sum / \max$ | Measured $S$ |
|---|---|---|---|---|
| 3+3 | 12.20 | 2.99 | 1.24 | 0.97 |
| **0+6** | **9.22** | **5.97** | **1.65** | **1.23** |

**Table 3:** Multiplications per token on each rank, the resulting upper bound on speed-up, and the measured speed-up against one node with two sequences.

9% slower than one, which isolates the network cost. Most of the gain appears at window 2, and beyond window 4 throughput no longer rises in a meaningful way, because additional texts only wait in the queue. The worst run with window 4 (74.7 s) is faster than the best run on one node (96.9 s).

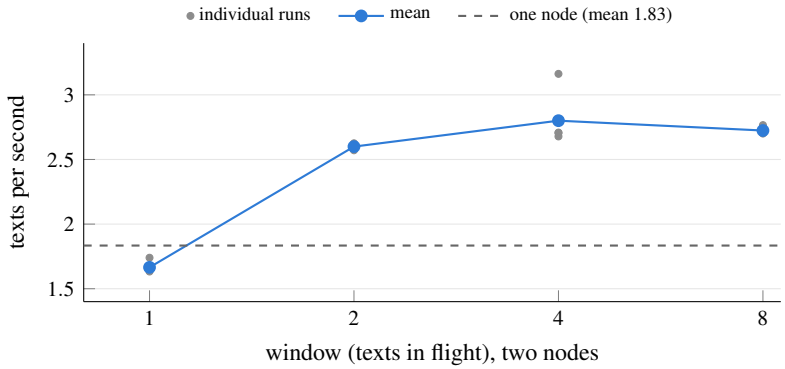



**Figure 5:** `all-MiniLM-L6-v2` on two nodes as a function of the window, 200 texts. Gain requires simultaneous work; a larger window does not add much.

### 5.3 Scaling on four nodes

Table 4 and Figure 6 give the main result, for `bge-base-en-v1.5`. Throughput increases at each step: 1.39 times on two nodes and 2.16 times on four. The ranges of the runs do not overlap, so the ordering is not due to run-to-run variation. Efficiency falls from 69% to 54%, which means each added node gives less. Without the first run of each configuration, which on one node was also the slowest, $S$ on four nodes is about 2.0.

Contrary to what we expected, the larger model did not improve efficiency. On two nodes its speed-up is 1.39, against 1.53 for MiniLM, although the two runs used different texts and the numbers are not strictly comparable.

| Nodes | Runs (s) | Mean (s) | Texts/s | $S$ | $E$ |
|---|---|---|---|---|---|
| 1 | 321.9, 273.1, 263.3 | 286.1 | 0.175 | – | – |
| 2 | 234.8, 194.4, 189.4 | 206.2 | 0.242 | 1.39 | 69% |
| **4** | **126.4, 143.2, 128.0** | **132.6** | **0.377** | **2.16** | **54%** |

**Table 4:** `bge-base-en-v1.5` on 50 texts (3,137 tokens), with windows of 4 and 8 on two and four nodes.

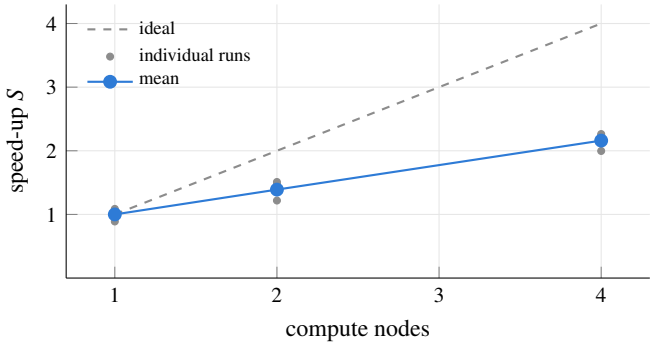



**Figure 6:** Speed-up of `bge-base-en-v1.5` against the mean time on one node.

### 5.4 Where the time goes

Indexing all 1,000 texts on four nodes took 2,151.8 s, or 0.46 texts per second. Figure 7 shows that the work is well balanced: each node computes for about 1,600 s and spends the remaining 25–26% in MPI calls, on transfers and idle waiting.

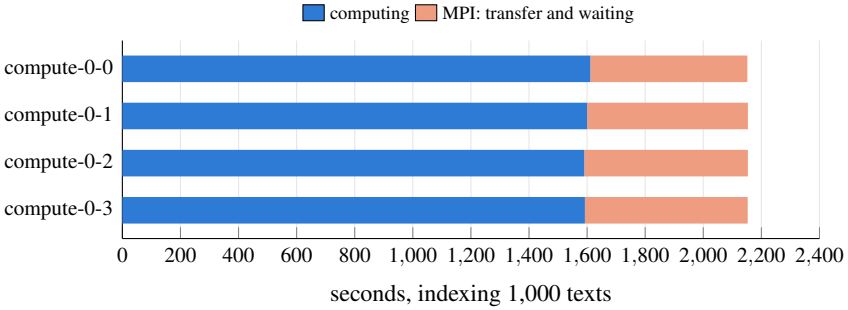



**Figure 7:** Time per rank for indexing with `bge-base-en-v1.5`, window 8.

The loss in efficiency can be divided roughly into waiting and into slower computing. Let $U$ be the fraction of time that ranks compute, and $\kappa$ the total computing time on $N$ nodes divided by $T_1$. Then

$$S = \frac{NU}{\kappa}. \tag{6}$$

Here $U = 0.74$. Extrapolating $T_1$ from the single-node rate of Table 4 gives about 4,966 s for this workload, so $\kappa \approx 1.29$ and $S \approx 2.31$. The same model needs 29% more computing time when four VMs run together, which suggests contention for the laptop's cores, memory and cache. This is an estimate: it compares different texts and was not isolated experimentally.

The virtual network is slow compared with memory. The one-way latency for a 3 KB message was 3.2–3.8 ms between compute nodes, while two processes on one VM measured 0.7 $\mu$s for a 1.2 KB message. Messages between stages are not large, however: an average text of about 60 tokens needs about 180 KB per hop.

### 5.5 Memory

Table 5 shows that splitting reduces the memory of each node. Rank 0 keeps the embedding table, the largest single tensor, so its share is the largest. Measured memory can be lower than the assigned weights, because

only pages actually touched are loaded through `mmap`, and most of the vocabulary is not used. All three models would fit on one 2 GB node, so this work demonstrates the mechanism rather than solving a real memory limit.

| Model and run | Node | Weights assigned (MB) | Peak memory (MB) |
|---|---|---|---|
| `stories15M`, 1 node | – | 58.0 | 66.9 |
| `stories15M`, split 3+3 | rank 0 / rank 1 | 46.6 / 11.4 | 54.4 / 17.4 |
| `stories15M`, split 0+6 | rank 0 / rank 1 | 35.2 / 22.8 | 41.3 / 33.8 |
| `all-MiniLM`, split 3+3, 1,000 texts | rank 0 / rank 1 | 65.8 / 20.3 | 56.0 / 28.3 |
| `bge-base`, 4 nodes, 1,000 texts | rank 0 / ranks 1–3 | 172.0 / 81.1 | 136.5 / 92.9 |

**Table 5:** Weights assigned to each rank and peak resident memory.

### 5.6 Live view and semantic search

Figure 8 shows a real run of the live view. The time intervals overlap: from about 21.95 s to 26.26 s, ranks 1, 2 and 3 work simultaneously on texts 7, 6 and 5. This is the behaviour sketched in Figure 1, here on real nodes.

```
[  17.063 -   21.709 s] rank 2 compute-0-2.local | text 5 | 182 tokens | layer
6-8 | 4646.5 ms | sends 546.0 KB to rank 3
[  21.882 -   26.260 s] rank 2 compute-0-2.local | text 6 | 172 tokens | layer
6-8 | 4377.7 ms | sends 516.0 KB to rank 3
[  21.952 -   26.511 s] rank 1 compute-0-1.local | text 7 | 182 tokens | layer
3-5 | 4558.6 ms | sends 546.0 KB to rank 2
[  21.813 -   26.552 s] rank 3 compute-0-3.local | text 5 | 182 tokens | layer
9-11 | 4739.0 ms | sends 3.0 KB to rank 0
[  26.578 s] rank 0 compute-0-0.local | text 5 | final vector received
[  26.634 -   30.315 s] rank 3 compute-0-3.local | text 6 | 172 tokens | layer
9-11 | 3681.1 ms | sends 3.0 KB to rank 0
[  30.342 s] rank 0 compute-0-0.local | text 6 | final vector received
[  26.709 -   30.539 s] rank 2 compute-0-2.local | text 7 | 182 tokens | layer
6-8 | 3830.4 ms | sends 546.0 KB to rank 3
[  30.644 -   34.123 s] rank 3 compute-0-3.local | text 7 | 182 tokens | layer
9-11 | 3478.4 ms | sends 3.0 KB to rank 0
[  34.137 s] rank 0 compute-0-0.local | text 7 | final vector received

--- ranks: 4 | window: 8 | texts: 8 | tokens: 819 | time: 34.099 s | throughpu
 0.23 texts/s
--- one-way latency rank 0 <-> 1, 3072 bytes: 3775.1 us
rank host                 layers  compute (s)  MPI+wait (s) weights (MB) peak
S (MB)
0    compute-0-0.local    0-2          20.037        14.021        172.0
  97.5
1    compute-0-1.local    3-5          20.805        13.297         81.1
  92.8
2    compute-0-2.local    6-8          20.641        13.474         81.1
  92.8
3    compute-0-3.local    9-11         19.694        14.425         81.1
  92.8
[mihai@headnode embed]$
```

**Figure 8:** Output of `live.sh` on four nodes: each line gives a rank's interval, text, tokens, layers and transfer, followed by the summary per rank.

For search, the 1,000 stored vectors are compared with a question's vector. As recommended by the model's authors, questions get the prefix "Represent this sentence for searching relevant passages:". Table 6 and Figure 9 show results. A question crosses all four stages, each taking about 300 ms, so embedding takes 1.3–1.5 s; scanning the vectors takes under a millisecond. The first question after the long indexing run took 54.9 s, probably because the model was no longer cached on the nodes, though we did not measure this separately. A short warm-up question avoids this delay in a demonstration.

A single question is in fact slower on the cluster. On one node, a nearly identical question, one word longer, took 0.65 s once the model was loaded, against 1.34 s on four nodes. So one question is about twice as slow on four nodes. Each hop adds only about 20–30 ms in the live view, so most of the difference seems to come from slower computing per stage: about 300 ms for three layers on four nodes, against about 160 ms if the 0.65 s is divided evenly over four groups of layers. This rests on a single measurement.

| Question | Tokens | Best match (score) | Embed (s) | Search (ms) |
|---|---|---|---|---|
| *oil prices are rising* | 14 | Oil prices surge to a new high (0.729) | 1.34 | 0.6 |
| *new computer chips from Intel* | 15 | Intel in new chip breakthrough (0.683) | 1.38 | 0.6 |
| *who won the footbal match* [sic] | 16 | US edge out Brazil for gold (0.611) | 1.52 | 0.5 |

**Table 6:** Interactive questions on four nodes. Tokens include the prefix, [CLS] and [SEP].

```
oil prices are rising
[   5.996 -    6.310 s] rank 0 compute-0-0.local | text 0 | 14 tokens | layers 0
-2 | 314.5 ms | sends 42.0 KB to rank 1
[   6.329 -    6.651 s] rank 1 compute-0-1.local | text 0 | 14 tokens | layers 3
-5 | 322.8 ms | sends 42.0 KB to rank 2
[   6.681 -    6.993 s] rank 2 compute-0-2.local | text 0 | 14 tokens | layers 6
-8 | 312.2 ms | sends 42.0 KB to rank 3
[   7.010 -    7.335 s] rank 3 compute-0-3.local | text 0 | 14 tokens | layers 9
-11 | 324.6 ms | sends 3.0 KB to rank 0
[   7.360 s] rank 0 compute-0-0.local | text 0 | final vector received
  0.729  Oil prices surge to a new high WASHINGTON -- The price of oil charged t
o a new high above  $47 a barrel yesterday amid nagging concerns about instabili
ty in Iraq, the uncertain fate of Russian petroleum giant Yukos, and the world's
 limited supply cushion.
  0.721  Oil prices bubble to record high The price of oil has continued its sha
rp rise overnight, closing at a record high. The main contract in New York, ligh
t sweet crude for delivery next month, has closed at a record  $US46.75 a barrel
 - up 70 cents on yesterday #39;s close.
  0.716  Oil Hits New High Over  $48 as Iraq Violence Flares LONDON (Reuters) -
Oil prices struck a fresh record above  $48 a barrel on Thursday, spurred higher
 by renewed violence in Iraq and fresh evidence that strong demand growth in Chi
na and India has not been slowed yet by higher energy costs.
  0.703  Will High Oil Prices Lead to Recession? (AP) AP - High oil prices, whic
h have been a factor in virtually all U.S. recessions over the past three decade
s, are surging again this year. And the higher crude oil prices climb, the more
 risk energy costs pose to what, until recently, many expected to be a banner yea
r for the U.S. economy.
  0.700  Oil Prices Alter Direction After a month-long rally that repeatedly pus
hed prices to new highs, the cost of a barrel slumped for the fourth day, leavin
g the price  $10 higher than year-ago rate.
  (1369.0 ms to embed, 0.8 ms to search)

[mihai@headnode embed]$
```

**Figure 9:** A search on four nodes: the question crosses ranks 0 to 3, and the most similar news texts are returned with their scores.

## 6 Limitations

- All VMs share one laptop, so the nodes are not independent machines. Its cores are not identical, and the host's scheduling is outside our control. With the other VMs stopped, two processes on the head node ran `stories15M` at 65.9 tokens per second, against about 17–19 on the compute nodes; this difference was not investigated.
- Each configuration has only three or four runs, and variation is visible. One-node BGE runs took between 263.3 and 321.9 s. No confidence intervals are reported.
- The MiniLM window 4 mean includes a separate run of 63.2 s, well below its other three of about 74 s. Without it, the gain is about 1.47 times instead of 1.53.
- Transfer time and idle waiting were not separated, and three nodes were not measured.
- The C tokenizer was tested on English and a few accented and non-Latin examples, not on all languages.

## 7 Conclusions

Pipeline parallelism on a small virtual CPU cluster can work, but only under specific conditions. With independent texts, a balanced split and a window larger than one, `bge-base-en-v1.5` reached 2.16 times the throughput of one node on four nodes, with outputs that match the reference implementation. Under different conditions the same technique does not help. Sequential generation of one text became 23% slower, and an even split of `stories15M` gained nothing, because its classifier outweighs all its layers. Latency for a single question also does not improve: as every question must cross all nodes, it took 1.34 s on four nodes against 0.65 s on one, in a single measurement.

Natural next steps are repeating the measurements on physical machines, separating transfer from waiting, and pinning each VM to its own physical cores. A model that does not fit in one node's memory would also test the case that motivates splitting in practice.

## References


[1] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems*, volume 32, pages 103–112, 2019.

[2] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. PipeDream: Generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*, pages 1–15, 2019.

[3] Philip M. Papadopoulos, Mason J. Katz, and Greg Bruno. NPACI Rocks: Tools and techniques for easily deploying manageable Linux clusters. *Concurrency and Computation: Practice and Experience*, 15(7–8):707–725, 2003.

[4] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, 2023.

[5] Message Passing Interface Forum. MPI: A message-passing interface standard, version 4.1. `https://www.mpi-forum.org/docs/`, 2023.

[6] Edgar Gabriel, Graham E. Fagg, George Bosilca, Thara Angskun, Jack J. Dongarra, Jeffrey M. Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, Ralph H. Castain, David J. Daniel, Richard L. Graham, and Timothy S. Woodall. Open MPI: Goals, concept, and design of a next generation MPI implementation. In *Proceedings of the 11th European PVM/MPI Users' Group Meeting*, pages 97–104, 2004.

[7] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.

[8] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4171–4186, 2019.

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[10] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). arXiv preprint arXiv:1606.08415, 2016.

[11] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. arXiv preprint arXiv:1607.06450, 2016.

[12] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[13] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Maksim Riabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative inference and fine-tuning of large models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 558–568, 2023.

[14] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538, 2022.

[15] Andrej Karpathy. llama2.c: Inference Llama 2 in one file of pure C. `https://github.com/karpathy/llama2.c`, 2023.

[16] Hugo Touvron et al. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288, 2023.

[17] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? arXiv preprint arXiv:2305.07759, 2023.

[18] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[19] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general Chinese embeddings. arXiv preprint arXiv:2309.07597, 2023.

[20] Xiang Zhang, Junbo Zhao, and Yann LeCun. Character-level convolutional networks for text classification. In *Advances in Neural Information Processing Systems*, volume 28, 2015.